

◆ PRODUCT REQUIREMENTS

KINETIC TYPE

Product Requirements Document

Next-generation adaptive, domain-driven typing platform — replacing generic word-stream tests with real-world skill acquisition and targeted biomechanical improvement.



VERSION

1.0

STATUS

Draft for Review

DATE

31 Aug 2026

OWNER

Product — Kinetic Type

Confidential & Proprietary

Table of Contents

01	Executive Summary	03	02	Problem Statement	03
03	Target Users & Personas	04	04	Goals & Success Metrics	05
05	Product Scope (v1)	05	06	Functional Requirements	06
07	User Stories & Acceptance Criteria	07	08	Non-Functional Requirements	08
09	Assumptions & Dependencies	08	10	Risks & Mitigations	09
11	Release Criteria	09	12	Appendix – Glossary	10

This document defines *what* Kinetic Type must do and *why*. For implementation architecture see the Technical Requirements Document; for visual system see the UI/UX Brief; for phased delivery see the Implementation Plan.

01 Executive Summary

Kinetic Type is an adaptive, domain-driven typing platform designed to replace generic word-stream testing with real-world skill acquisition and targeted biomechanical improvement. Instead of measuring arbitrary speed on isolated dictionary words, it continuously profiles a typist's **sub-key latency**, detects **micro-flaws in key-pair transitions (n-grams)**, and measures **context-switching overhead** across specialized domains — code syntax, terminal commands, and literature/prose.

The product's core insight is that speed is a downstream symptom, not a target. A typist's ceiling is set by the slowest, error-prone finger transitions buried inside their normal typing — a slow `Shift+{` in code, a fumbled `sudo ↵` in a terminal, a hesitant `th` in prose. Kinetic Type finds those transitions automatically and rewrites the practice material in real time to target them, closing the loop between diagnosis and drill without the user ever configuring a lesson plan.

3

DOMAIN MODES AT LAUNCH

<1ms

LATENCY CAPTURE PRECISION

100%

FUNCTIONAL OFFLINE

0

MANUAL LESSON CONFIG

02 Problem Statement

Existing typing-improvement tools (MonkeyType, 10FastFingers, TypeRacer, Keybr) share three structural gaps that Kinetic Type is built to close:

GAP 01 — GENERIC CONTENT

Word streams ignore real work

Practice text is drawn from common-word corpora, not the symbol-dense, camelCase, bracket-heavy text developers and operators actually produce daily.

GAP 02 — AGGREGATE METRICS ONLY

WPM hides the real bottleneck

A single speed number cannot reveal *which* two-key transition is costing the user 40ms every time it appears — the actual lever for improvement.

GAP 03 — STATIC DIFFICULTY

No adaptive feedback loop

Practice sets don't evolve with the user; the same weak transitions persist session after session because nothing is engineered to reintroduce them.

OPPORTUNITY

Biomechanics as a first-class metric

Sub-millisecond key event timing is available in every browser via `performance.now()` and is simply unused by the category today.

PRODUCT THESIS

If we can detect a user's specific mechanical weak points and continuously feed them targeted, contextually realistic text, measurable improvement happens faster than with undifferentiated practice — and the user never has to know the system is doing it.

03 Target Users & Personas

Kinetic Type serves three primary personas at launch, unified by one trait: their day-to-day text is not conversational prose, so generic typing tools under-serve them.

PERSONA A — PRIMARY

"Dev Dana" — Software Engineer

Writes TypeScript/Python for 6+ hours a day. Fumbles on `=>`, `{ }` nested brackets, and `camelCase` / `snake_case` switches under Shift. Wants a warm-up ritual that measurably reduces mid-session typos in real code, not a vanity WPM score.

PERSONA B — PRIMARY

"Ops Omar" — SRE / DevOps Engineer

Lives in a terminal: flag-heavy commands, regex, absolute paths. A single mistyped flag can be costly. Wants drilling on exact command patterns he uses, with error-rate reduction as the headline metric, not speed.

PERSONA C — SECONDARY

"Prose Priya" — Writer / Transcriptionist

Long-form prose and transcription work. Cares about sustained throughput and consistency (avoiding fatigue-driven slow-downs) more than peak burst speed.

Jobs-to-be-done

PERSONA	JOB - TO - BE - DONE	CURRENT ALTERNATIVE	WHY IT FALLS SHORT
Dev Dana	"Help me stop fumbling symbols under time pressure while coding."	MonkeyType "code" mode	Generic snippets, no per-transition diagnostics
Ops Omar	"Build muscle memory for the exact commands I run."	Manual repetition / aliases	No feedback loop, no measurement
Prose Priya	"Improve sustained accuracy across a long writing session."	10FastFingers	Optimizes burst speed, ignores consistency & fatigue

04 Goals & Success Metrics

OBJECTIVE	KEY RESULT / KPI	V1 TARGET
Prove the adaptive loop improves outcomes	Median reduction in avg. latency of a user's top-10 weakest key-pairs after 14 days of use (≥ 3 sessions/week)	$\geq 20\%$
Deliver true domain relevance	% of session text judged "realistic to my work" (in-product survey)	$\geq 80\%$
Keep the instrument invisible	Input-to-render latency introduced by the app itself	< 5 ms p95
Retain through habit formation	D7 retention	$\geq 35\%$
Zero-friction first session	Time from landing to first completed session	< 60 sec
Data trust	Sessions with zero data loss (local persistence integrity)	99.9%

05 Product Scope — V1

✓ IN SCOPE

- Dev, CLI, and Prose/Transcribe domain modes
- Sub-millisecond key-pair latency capture & n-gram matrix
- Dynamic Material Synthesizer (adaptive text generation)
- Custom zero-latency canvas/DOM text renderer
- Local-first persistence (works fully offline)
- Visual analytics: latency heatmap, micro-pause/fatigue graphs
- Optional account creation with cross-device sync
- Session history & trend view

✗ OUT OF SCOPE (V1)

- Mobile / touch input support
- Multiplayer races or social competition
- Custom user-authored corpora / lesson editor
- Team/enterprise admin dashboards & seat management
- Non-QWERTY layout-specific tuning (Dvorak, Colemak)
- Voice or non-keyboard input modalities
- Native desktop/mobile apps (V1 is web-only, installable PWA)

06 Functional Requirements

ID	REQUIREMENT	PRIORITY
FR-01	Capture <code>keydown/keyup</code> timestamps via <code>performance.now()</code> for every keystroke to compute physical transit time between consecutive key pairs.	P0
FR-02	Maintain a persistent per-user Key-Pair Performance Matrix (avg latency, occurrence count, error count) across sessions.	P0
FR-03	Provide three built-in domain lexers at launch: Dev (code syntax), CLI (shell/terminal), Prose (literature/transcription).	P0
FR-04	Dynamically synthesize upcoming practice text by weighting the user's historically slowest/error-prone key pairs into the stream ($\geq 70\%$ targeted, $\leq 30\%$ contextual flow).	P0
FR-05	Render the typing surface with a custom virtualized renderer (not native form inputs) to eliminate browser input-lag and layout shift.	P0
FR-06	Compute and display gross WPM, net WPM (error-adjusted), and a burst-consistency score per session.	P0
FR-07	Detect micro-pauses (anomalous inter-key gaps) as a proxy for hesitation/fatigue and surface them post-session.	P1
FR-08	Render a key-pair latency heatmap (keyboard-shaped) summarizing a session or rolling window.	P1
FR-09	Persist all performance data locally (offline-first) with zero required network dependency to use the product.	P0
FR-10	Allow optional account creation to back up and sync performance history across devices.	P1
FR-11	Apply spaced-repetition scheduling logic so previously-weak, now-improved pairs periodically re-enter the stream to confirm retention.	P1
FR-12	Show a session summary screen with domain, WPM (gross/net), accuracy, and top-3 improved/worsened key pairs.	P0
FR-13	Allow users to select session length (time-based or character-count-based) before starting.	P2
FR-14	Provide a reduced-motion / high-contrast accessibility mode.	P1

07 User Stories & Acceptance Criteria

ID	STORY	ACCEPTANCE CRITERIA
US-01	As Dev Dana, I want to select "Dev Mode" so my practice text resembles real code.	Domain selector visible on start screen; Dev Mode generates nested blocks, multi-char operators, bracket pairs, camelCase/snake_case tokens.
US-02	As any user, I want the app to quietly learn my weak spots without configuring anything.	After session 1, ≥1 key pair enters the weighted pool automatically; no settings screen required to enable adaptivity.
US-03	As Ops Omar, I want CLI-flavored text with real flags and paths.	CLI Mode includes flags (<code>--force</code> , <code>-rf</code>), regex, absolute paths, piping.
US-04	As a returning user, I want to see whether I'm actually improving.	History view plots avg. latency of previously-flagged pairs over time, trending downward when practiced.
US-05	As a privacy-conscious user, I want to use the full product without an account.	All core functionality (typing, adaptive engine, analytics) works with zero sign-in, data stored locally only.
US-06	As a multi-device user, I want my history to follow me if I sign in.	Signing in syncs local IndexedDB history to the cloud and merges on a second device without data loss.
US-07	As any user, I want immediate, lag-free visual feedback per keystroke.	Perceived input latency introduced by rendering is imperceptible (<5ms, no native text-box caret).
US-08	As a user reviewing a session, I want to see exactly which transitions cost me the most time.	Post-session screen ranks top offending key pairs with ms values and occurrence counts.

DEFINITION OF "ADAPTIVE," MADE CONCRETE

A requirement is only "done" if it changes what text a real user sees on their *next* session based on what happened in a *previous* one — not merely logged, but acted upon by the synthesizer.

08 Non-Functional Requirements

CATEGORY	REQUIREMENT
Performance	Input-to-paint latency contributed by the app < 5ms p95; timestamp capture resolution sub-millisecond.
Reliability	No session data loss on crash/refresh — writes flushed to IndexedDB incrementally, not only at session end.
Availability	Core product fully functional with zero network connection.
Privacy	Keystroke <i>content</i> is never required to leave the device beyond what's needed for domain text generation; only aggregate timing/error metrics sync to the cloud.
Accessibility	WCAG 2.1 AA color contrast; reduced-motion mode; full keyboard operability of all non-typing UI.
Browser Support	Latest 2 versions of Chrome, Edge, Firefox, Safari (desktop).
Scalability	Cloud sync layer supports growth to 100K+ MAU without schema redesign.

09 Assumptions & Dependencies

- Users have a physical keyboard; touch/mobile is explicitly out of scope for V1.
- `performance.now()` resolution is sufficient across target browsers for sub-ms differential timing (relative deltas, not absolute precision, are what matters).
- Firebase (Auth, Firestore, Cloud Functions, Hosting) is the assumed cloud backend — see TRD.
- Domain corpora (code samples, CLI commands, literature excerpts) can be sourced/licensed for redistribution inside the app bundle.

10 Risks & Mitigations

RISK	IMPACT	MITIGATION
Browser timer throttling (background tabs, power-saving modes) skews latency data	High	Discard/flag samples when <code>document.visibilityState</code> changes; use relative deltas, not absolute clock reliance.
Adaptive synthesizer over-fits, producing unnatural/frustrating text	Medium	Hard cap targeted-word ratio at 70%; always blend in natural flow words; allow a "less aggressive" adaptivity setting.
Users perceive the product as "just another typing test"	Medium	Onboarding explicitly surfaces the adaptive diagnosis in the first session ("We found your 3 slowest transitions").
Local-only data lost on browser data clear	Medium	Prompt for optional account backup after a threshold of sessions; export-to-file fallback.
Domain corpus feels generic despite "realistic" claim	Medium	Source corpora from real open-source repos / man pages / public-domain literature, not synthetic text.

11 Release Criteria (V1 GA)

- All P0 functional requirements implemented and passing acceptance criteria.
- p95 input-to-paint latency < 5ms verified on target browser matrix.
- Zero data-loss verified across 500+ simulated session/crash cycles.
- Adaptive loop verified: synthetic test user's injected weak pairs measurably re-appear at higher frequency within 1 session.
- Accessibility audit passed (WCAG 2.1 AA).
- Firebase security rules reviewed and penetration-tested for the sync path.

12 Appendix — Glossary

TERM	DEFINITION
Key-pair (n-gram)	An ordered two-key transition, e.g. <code>e→r</code> or <code>Shift→{</code> , treated as the atomic unit of measurement.
Latency Matrix	The persistent store of every key-pair's rolling average transition time, occurrence count, and error rate.
Dynamic Material Synthesizer	The engine that assembles upcoming practice text by over-weighting the user's worst-performing key pairs.
Domain Mode	A content lexer profile (Dev, CLI, Prose) that determines the syntactic shape of generated text.
Micro-pause	An anomalous gap between keystrokes, statistically distinct from normal transition latency, used as a fatigue/hesitation signal.
Net WPM	Words-per-minute adjusted for uncorrected errors.

Related Documents

- TECHNICAL REQUIREMENTS DOCUMENT
- UI / UX BRIEF
- APP FLOW
- BACKEND SCHEMA
- IMPLEMENTATION PLAN

End of Product Requirements Document — Kinetic Type v1.0. Prepared 31 August 2026. Confidential & Proprietary.